

15/03/22

MODULE 4: Configuration Management

Introduction

- Developer uses the server with same configuration as production
- Realistic expectations about access and compatibility

- More automation - configuration management
- Developer and sysadmin help and educate one another (Chef, Jenkins)

- Distributed tasks mean fewer choke points (single person who can block task)



- Dev-Ops Collaboration
- Automation to achieve CD
- CD – CM/Provisioning
- Ops provide access to the servers with the same configuration as production.

- CM enables ops to define their infrastructure in *code*
- Install packages, configure software, start/stop services
- Ensure a state of a machine
- Ensure policies and standards are in place
- Provide history of changes for a system
- Repeatable way of rebuild a system
- Orchestrate a cluster of services together

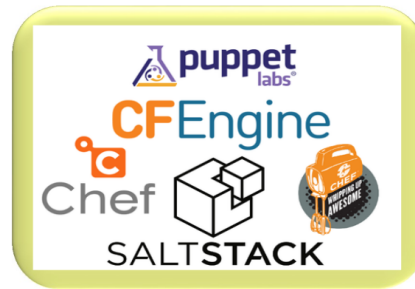
- Configuration management is the process of standardizing resource configurations and enforcing their state across IT infrastructure in an automated manner.

History of Configuration Management

➤ Configuration Management originated in the United States Department of Defense in the 1950s as a technical management discipline for hardware material items—and it is now a standard practice in virtually every industry.

➤ CM is the practice of handling changes systematically so that a system maintains its integrity over time.

➤ Continuous Configuration Automation tool are included in this realm of DevOps tooling and are often included as part of a DevOps toolchain.

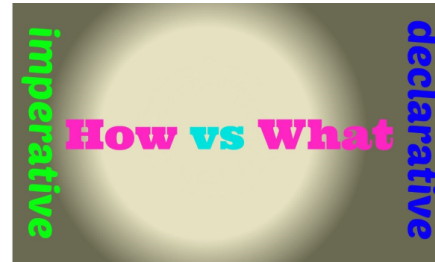


Configuration Management & Infrastructure as Code (IaC)

- The purpose of Software Configuration Management is to establish and maintain the **integrity of the products of the software project** throughout the project's software life cycle.
- Configuration management (CM) refers to a discipline for evaluating, coordinating, approving or disapproving, and **implementing changes in artifacts** that are used to construct and maintain software systems.
- An artifact may be a piece of hardware or software or documentation. (e.g., use cases, Unified Modeling Language (UML) models, requirements and design documents)
- CM is intended to **eliminate the confusion and error** brought about by the existence of different versions of artifacts.
- CM is about keeping the inevitable change under control.
- Without a well-enforced CM process, different team members (possibly at different sites) can use different versions of artifacts unintentionally;
 - individuals can create versions without the proper authority; and the wrong version of an artifact can be used inadvertently.

Declarative vs Procedural IaC

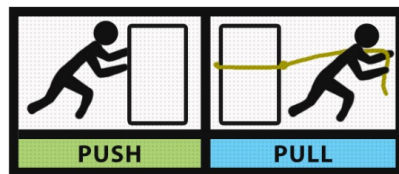
- There are generally two approaches to IaC:
 - declarative (functional)
 - vs.
 - imperative (procedural).



- The difference between the declarative and imperative approach is essentially **'what'** versus **'how'**.
- The declarative approach focuses on what the eventual target configuration should be; whereas, imperative focuses on how the infrastructure is to be changed to meet.
- The declarative approach defines the desired state and the system executes what needs to happen to achieve that desired state.
- Imperative defines specific commands that need to be executed in the appropriate order to end with the desired conclusion.

Clearly declarative approach generalizes better than functional.

Push vs Pull IaC



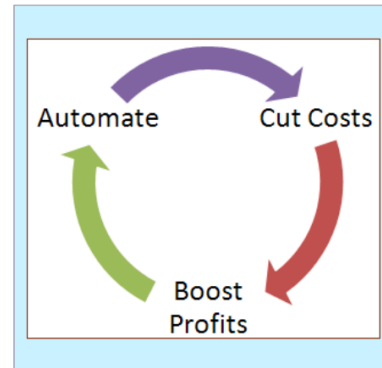
- There are two methods of IaC: *'Push'* and *'Pull'*.
- In the Pull method the server to be configured will pull its configuration from the controlling server.
- In the Push method the controlling server pushes the configuration to the destination system.

Pull method => Node that needs configuration pulls config from control server when the config/data is "ready"/new version of the config is available => Like Jenkins master pulling from remote repo via polling

Push method => Controlling server pushes config to the node that needs configuration when control server's data/config is "ready"/new version of the config is available => Like GitHub Webhook trigger with Jenkins job builds => **More scalable method for larger network.**

IaC Automation

- Automation in general aims to take the confusion and **error-prone aspect of manual processes** and make it more efficient, and productive.
- Allowing for better software and applications to be created with flexibility, less downtime, and an overall cost effective way for the company.



- IaC is intended to reduce the complexity that kills efficiency out of manual configuration.
- Automation and collaboration are considered central points in DevOps**
- Infrastructure automation tools are often included as components of a DevOps toolchain.



- There is no one product that can be considered a single DevOps tool.
- Configure activities fall under the operation side of DevOps.
- Once software is deployed, there may be additional IT infrastructure provisioning and configuration activities required.

- Specific activities including:
 - storage, database, network and application provisioning.
- The main types of solutions that facilitate these activities are
 - Continuous Configuration Automation, configuration management, and Infrastructure as Code tools.

Ansible is the meta right now, so Chef part is just for learning purposes.

Benefits of Configuration Management

- Continuous deployment is that every change is automatically deployed to production.
- In order to do continuous deployment one must be doing continuous delivery.

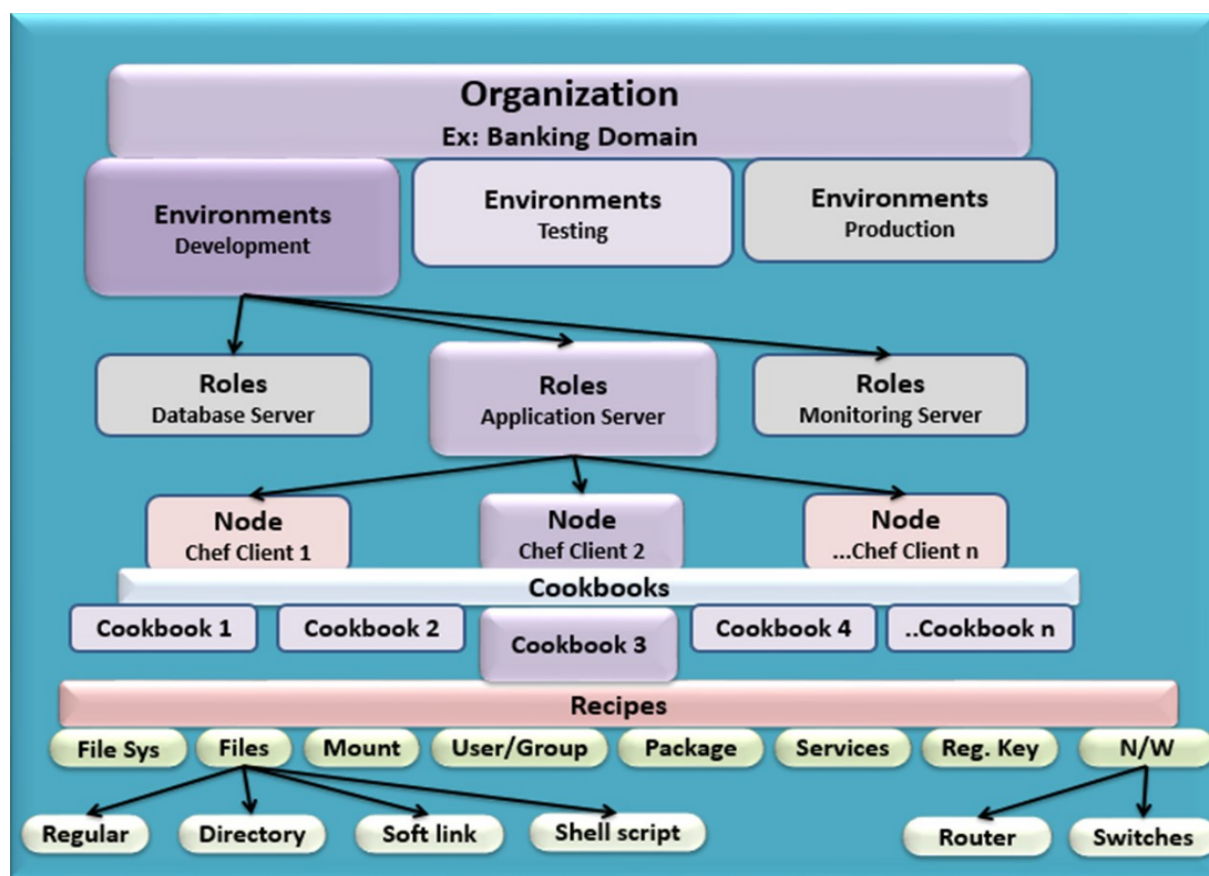
Several benefits of Configuration Management are:

- With CM, you can empower developers in Infrastructure as code
- It helps developers to get better understanding of what sort of infrastructure their applications need
- Increased ability to add new nodes in minutes to keep up with peaks or when system load is heavy
- Eliminate nearly all manual infrastructure management to significantly reduce management time and costs
- Ensure maximum system consistency with "infrastructure as code"
- Improve business agility with model for reuse

Docker -> DockerHub

Chef -> Market [lot of Kitchen wordplay going on over here]

Chef Architecture

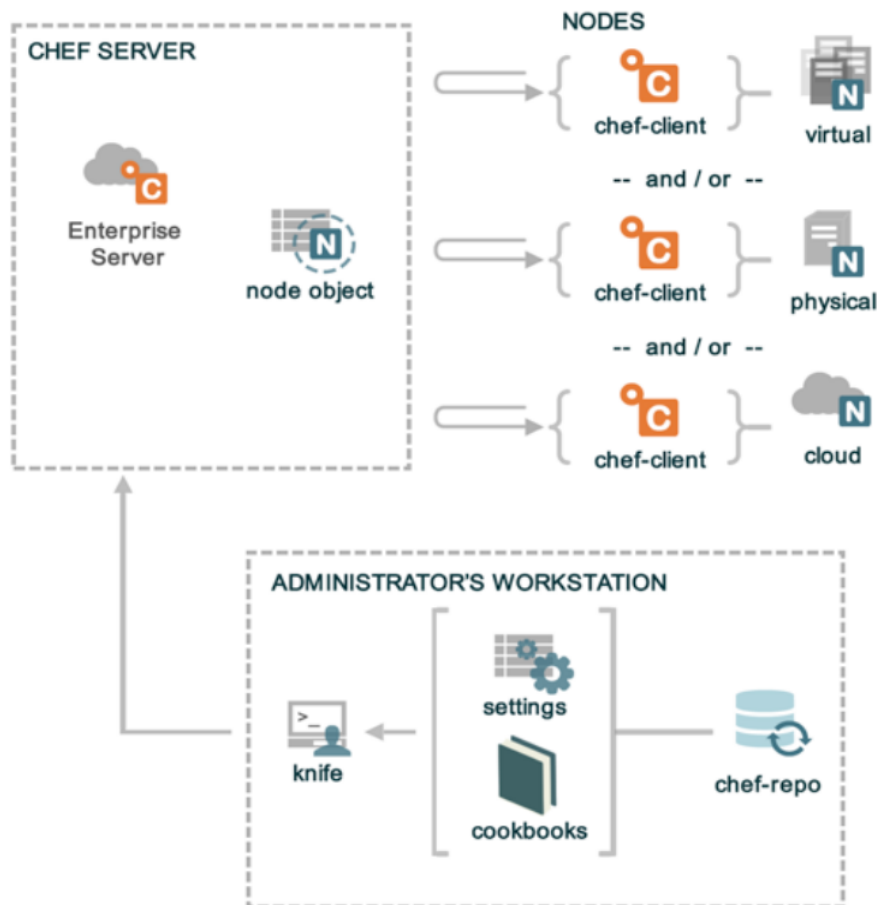


Each environment has different sets of roles and each role can have different sets of cookbooks.

Each cookbook contains recipes and each recipe dictates what is to be done or what is to be installed, etc.

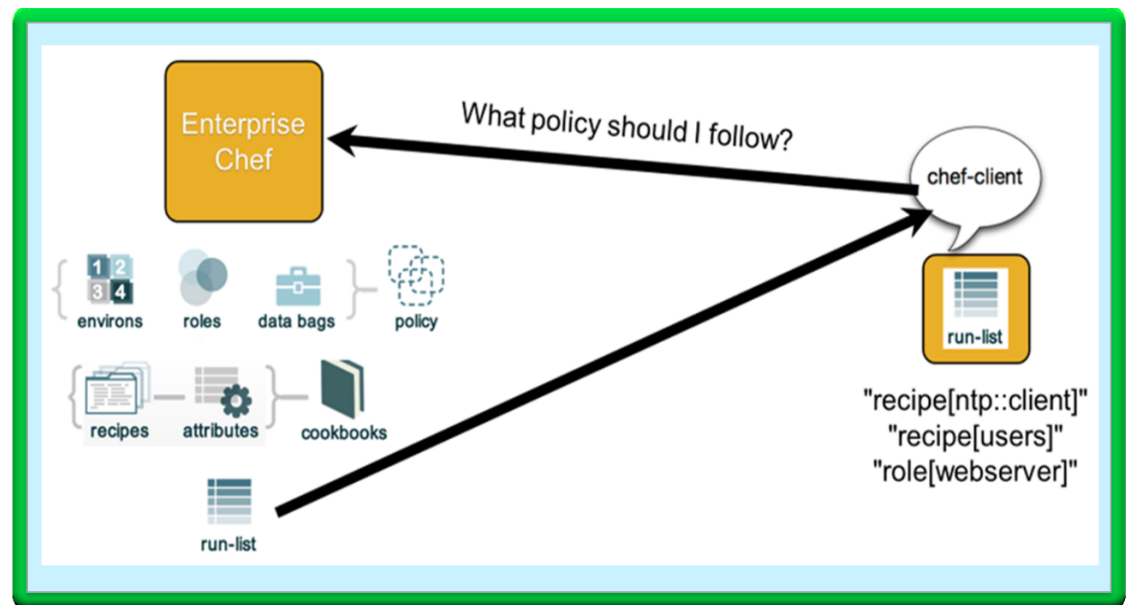
A cookbook will be shared with all nodes (Chef Clients) that supports the role type of the cookbook.

Chef Workflow



We have to register all our machines with the Chef server, i.e. their IP addresses.

Chef Clients



- Chef **data bags** can be defined as an arbitrary collection of data which one can use with cookbooks.
- **Attributes** in chef is a key-value pair which represents a specific detail about the node. For ex: If you want to get a ipaddress of each machine then I will create a one variable which will get it from each machine. It used by chef-client.

Cookbook Example

Problem: We need a web server configured to serve up our home page.

Success criteria: We can see the homepage in a web browser.

```
#
package "httpd" do
  action :install
end

service "httpd" do
  action [ :enable, :start ]
end

cookbook_file "/var/www/html/index.html" do
  source "index.html"
  mode "0644"
end
```

\$ knife cookbook upload apache

\$ knife node run_list add node1 "recipe[apache]"

apache = name of Cookbook
node1 = name of node
httpd = Apache Web Server library

Cookbooks contain Recipes which are **written in Ruby language**

For examples of beginner Recipes, you can check out sir's OpenSource4U article thingy

<https://www.opensourceforu.com/2016/03/chef-recipes-to-configure-the-desired-state-of-a-system/>

17/03/22: TA Session

GitHub Actions with nodeJS

Not just a CI/CD tool but instead automates entire developer workflows.

Features of GitHub Actions

Automating any Workflows

Built for open source projects and it's free

Automatically triggered by GitHub events such as PR, push, merge, issue created.

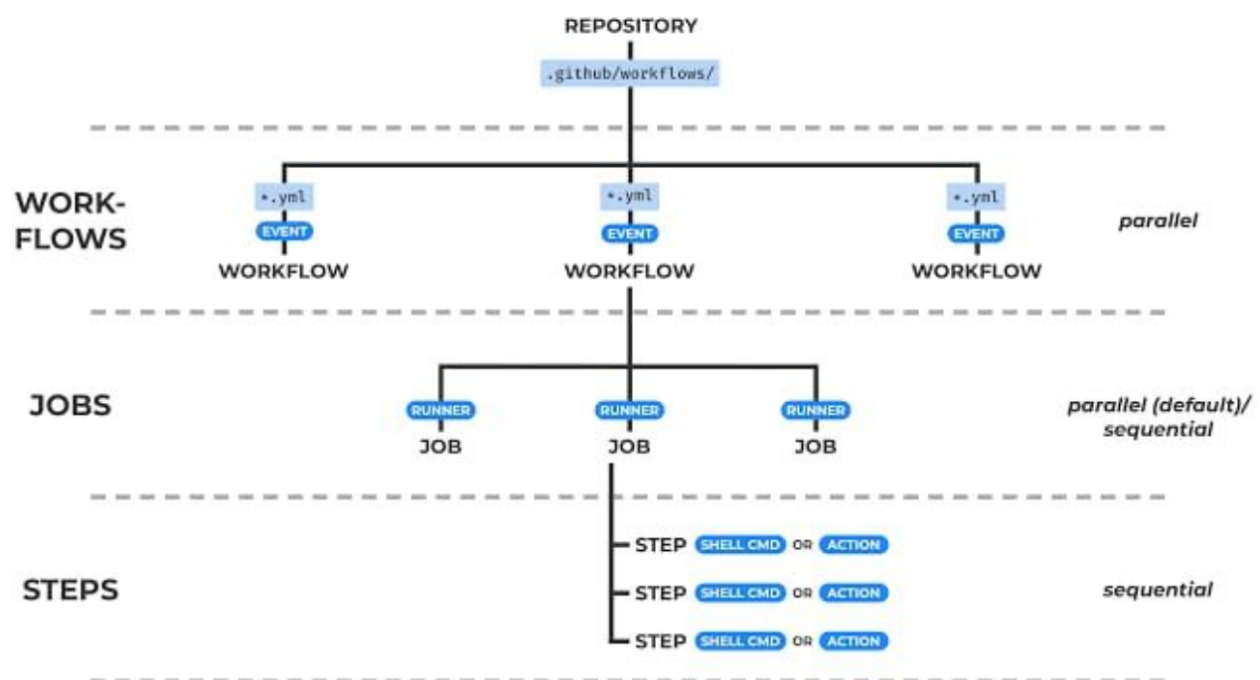
Trigger workflows based on branch specific actions and its developer friendly.

Integrate with different environments (supports many programming languages, tools like Docker, AWS, etc.)

Different plugins are available in the Marketplace.

No need of manual setup or does not require any VM [this is one of the key differentiators of GitHub Actions from Jenkins]

Note: Private repos have limited functionalities with GitHub Actions unless you have a Premium subscription or membership.



- A GitHub Action consists of Workflows which can run parallelly.
- A Workflow is triggered via an event and consists of jobs which also can run parallelly.
- There can be dependencies between Jobs, i.e. run Job A once Job B has finished executing, so this introduces sequential execution to Jobs.
- A job is a sequential list of steps.
- **Any yml file stored in the ".github" folder of the repository will be recognized as a Workflow, so each yml file is a separate parallelly executed Workflow.**

```
# This workflow will do a clean installation of node dependencies,
# cache/restore them, build the source code and run tests across different
# versions of node.
# For more information see:
https://help.github.com/actions/language-and-framework-guides/using-nodejs-with-github-actions

name: Node.js CI
# Build the job when there's a push or PR on master branch
on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]

jobs:
  # Name of job = build, which uses latest version of Ubuntu for OS
  # Various other options like windows-latest, ubuntu-16.04, etc. are also
  # there
  build:
    runs-on: ubuntu-latest
    # Define matrix strategy which enables you to run multiple jobs in
    # parallel, in this case, it is to build the same job but for
    # different versions of node, 12.x, 14.x, 16.x
    strategy:
      matrix:
        node-version: [12.x, 14.x, 16.x]
    # See supported Node.js release schedule at
    # https://nodejs.org/en/about/releases/
    # Define the steps that are involved in this job
    steps:
      # checkout is a GitHub Marketplace plugin that enables you to
      # checkout this GitHub repo, various options are available but we
      # are just checking out the entire repo with its current version only
      - uses: actions/checkout@v2
```

```
- name: Use Node.js ${{ matrix.node-version }}

# Installs node basically and sets the Node environment properly
uses: actions/setup-node@v2
with:
  node-version: ${{ matrix.node-version }}
  cache: 'npm'
  - run: npm ci
  - run: npm run build --if-present
```

npm ci does a clean install of all packages and dependencies defined in *package.json*

npm test is running the “test” script defined in *package.json*

needs is used to define job dependencies

CI tools stop whenever an error is encountered.

You define what is to be done when such a situation happens.

GitHub Actions sends you mail by default in such cases.

For the repo with this yml Workflow, if you push to the remote repo you can see everything in the GitHub Actions tab.

Points to Note

- By default, jobs have no dependencies and run in parallel with each other.
- To add dependencies, use *needs*:
- Use Build Matrix for faster deployments.
- Build your own customized pipelines faster and easier.
- Leverage Docker and Github actions.
- 3rd party Marketplace.

Mini Project

Make a Java calculator (menu-driven), add one function to it at a time and make the entire CI/CD pipeline which ends at Docker deployment.

Use GitHub Actions with either mini project or final and Jenkins with the other project.

22/03/22

Maven

Java Library Management framework.

Also helps assign proper directory structure to a Java project.

Why?

To manage Java projects that aren't dependent on any IDE.

Standard Directory Structure

Directory	Description
src/main/java	Your java source code
src/main/resources	Properties/config files used by your app
src/main/webapp	JSP files and web config files other web assets (image, css, js)
src/test	Unit testing code and properties
target	Destination directory for compiled code. Automatically created by Maven

Why Maven?

Creating a Java project

- Maven takes the group of assorted Java class files and resources, etc. and groups them into one thing called a project and the way Maven does this is by using the Project Object Model (POM).
- It's going to tell you where the source code is going to lie, where the test code goes, etc.

Adding dependencies manually

- All you have to do is mention the dependencies you require in the pom.xml file and Maven will take care of the rest.
- When Maven retrieves a project dependency, it will also download the supporting dependencies.
- So even in the later stages of the project, even if you need to upgrade some s/w, Maven will handle all the problems elegantly. Even if the version of some s/w changes, all you need to do is update the pom.xml, i.e. the main config file!

Automating the build process

- Maven provides these tasks (goals and targets) which allows you to build, compile and deploy, etc.
- It comes with a task runner and you can mention the order in which you want the tasks to run.
- When you build and run your app, Maven will handle the class/build path for you.
- Based on the config file, Maven will add JAR files accordingly.

Maven Characteristics

1. Convention over Configuration: It means that there is a certain convention that you can follow when you're working with Maven so that you don't have to tell it a whole lot. As long as you follow the convention, Maven knows all the necessary information about the Java project. But it allows you to configure it as well.

2. Consistent: That convention makes it consistent. You can go from one Maven project to another without much difficulty because they're mostly following the same convention.

3. It can still be configurable.

For eg: Maven expects source code to be in src/main/java, but you can configure it to look for somewhere else too.

4. It is IDE agnostic -- you don't need an IDE to use Maven. You can do pretty much everything without an IDE.

5. Even though it is IDE agnostic, it is IDE supportive. Pretty much all Java IDEs have support for Maven.

6. Declarative dependency management: For example, if you need apache commons, you declare it in pom.xml and Maven will make it available in your classpath. All you have to do is declare your demands!

7. Plugin based architecture: Most of Maven's features are plugin-based which makes it extensible and open to adding more features to Maven.

POM File

POM File Structure

Project metadata	Project name, version etc. Output file type: JAR, WAR,...
Dependencies	List of project we dependent on Spring, Hibernate etc
Plugins	Additional custom tasks to run: generate JUnit test reports etc

Project Coordinates

- Project Coordinates uniquely identify a project.

- Precise information for finding details about your project comes under project metadata in POM file structure.

Name	Description
GroupID	Name of company, group or organization Convention is to use reverse domain name: com.iiitb
Artifact ID	Name of the project: mavenlab
Version	A specific release version like 1.0, 1.6, 2.5.. If project is under active development then: 1.0-SNAPSHOT

Maven Workflow

- Read config file (pom.xml)
- Maven will check the local repository (~/.m2 folder) that resides on your machine.
- If not found it will check the central Maven repository (remotely hosted on the web) for those dependencies.
- Save the file in the local repo.
- Use those dependencies to build and run your application.

Example POM File

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.iiitb</groupId>
  <artifactId>MavenTutorial</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
```

```

        <version>4.12</version>
        <scope>test</scope>
    </dependency>
</dependencies>
</project>

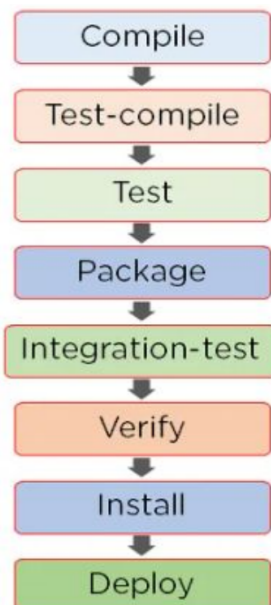
```

- This POM file compiles using Java 11 and installs JUnit as a dependency.
- <scope> is used to indicate to what extent is this dependency going to be used for.
- Like how we mention devDependencies with Node projects, here we can set 6 different values for scope, one of them is test and we use that here because JUnit is a testing library.

Maven Build Cycle Phases

- The Maven build life cycle is a sequence of steps that need to be followed in order to build a project, which eventually helps to execute goals.
- There are three types of standard life cycles: Clean, Default, Site.
 - clean: The clean life cycle handles project cleaning. It cleans up artifacts created by prior builds.
 - default: The default or build life cycle is the primary cycle that is responsible for building the application. There are 21 different phases in this primary life cycle, starting from the process of validation to the step of deployment.
 - site: The Maven site plugin handles the project site's documentation, which is used to create reports, deploy sites, etc.

Maven & DevOps



- **validate**: validate the project is correct and all necessary information is available
- **compile**: compile the source code of the project
- **test**: test the compiled source code using a suitable unit testing framework. These tests should not require the code be packaged or deployed
- **package**: take the compiled code and package it in its distributable format, such as a JAR.
- **integration-test**: process and deploy the package if necessary into an environment where integration tests can be run
- **verify**: run any checks to verify the package is valid and meets quality criteria
- **install**: install the package into the local repository, for use as a dependency in other projects locally
- **deploy**: done in an integration or release environment, copies the final package to the remote repository for sharing with other developers and projects.

So basically Maven is a build automation tool and it helps DevOps in providing automation around the Build phase of the DevOps Lifecycle Management

Maven Demo

```
sudo apt update
```

```
sudo apt install maven
```

```
mvn -version
```

```
sudo update-alternatives --config java          // Switch to Java 8 as there's less issues
```

Create a Maven project in IntelliJ with Java 11 or 8 (either works).

Add a new Java file under src/main/java and put something like "Hello World" or something in it.

Go to cmd and run "mvn install" in the directory where you have the pom.xml file placed.

Then from that same path run,

```
java -cp target/<JAR_file_name> HelloWorld // HelloWorld = Name of main class in the java file
```

```
java -cp target/<JAR_file_name> calculator/Calculator
```

// Because Calculator main class is stored in folder called calculator within the src/main directory

Integrating Maven with Jenkins + Pipeline Scripts + Testing

<https://nehakothari.notion.site/nehakothari/Maven-Demo-11c304c82c854216ab0dae35201faa>

24/03/22

Ansible

Why Ansible?

Problems/Issues

Manual deployment of infrastructure configuration, softwares, code modules, etc.

Infrastructure compatibility issues.

Security issues while connecting to hosts for deployment.

Solution: Ansible

- Ansible is an IT automation tool. It can configure systems, deploy software and orchestrate more advanced IT tasks such as continuous deployments or zero downtime rolling updates.

- Ansible's main goals are simplicity and ease-of-use. It also has a strong focus on security and reliability, featuring a minimum of moving parts, usage of OpenSSH for transport (with other transports and pull modes as alternatives), and a language that is designed around auditability by humans – even those not familiar with the program.

Implementation

1. Using shell commands
Manually writing shell scripts in order to execute both ad hoc commands as well as playbooks.
2. Using Jenkins pipeline
Creating a pipeline to automate git pull, maven build, and ansible deployment (Continuous Deployment)

Key Takeaways

- Ansible is agentless, meaning that it does not install software on the nodes that it manages. This removes a potential point of failure and security vulnerability and simultaneously saves system resources.
- Ansible is decentralized – it relies on your existing OS credentials to control access to remote machines.
- Relevant to all sizes of environments, users of all types: developers, sysadmins, release engineers, IT managers, and everyone in between. Ansible is appropriate for managing all environments, from small setups with a handful of instances to enterprise environments with many thousands of instances.

Why integrate Ansible with Jenkins?

- Jenkins allows us to perform all the pipeline stages in one go. No need to manually enter shell commands.
- Ease of writing pipeline syntax.
- Efficient debugging by checking the console output

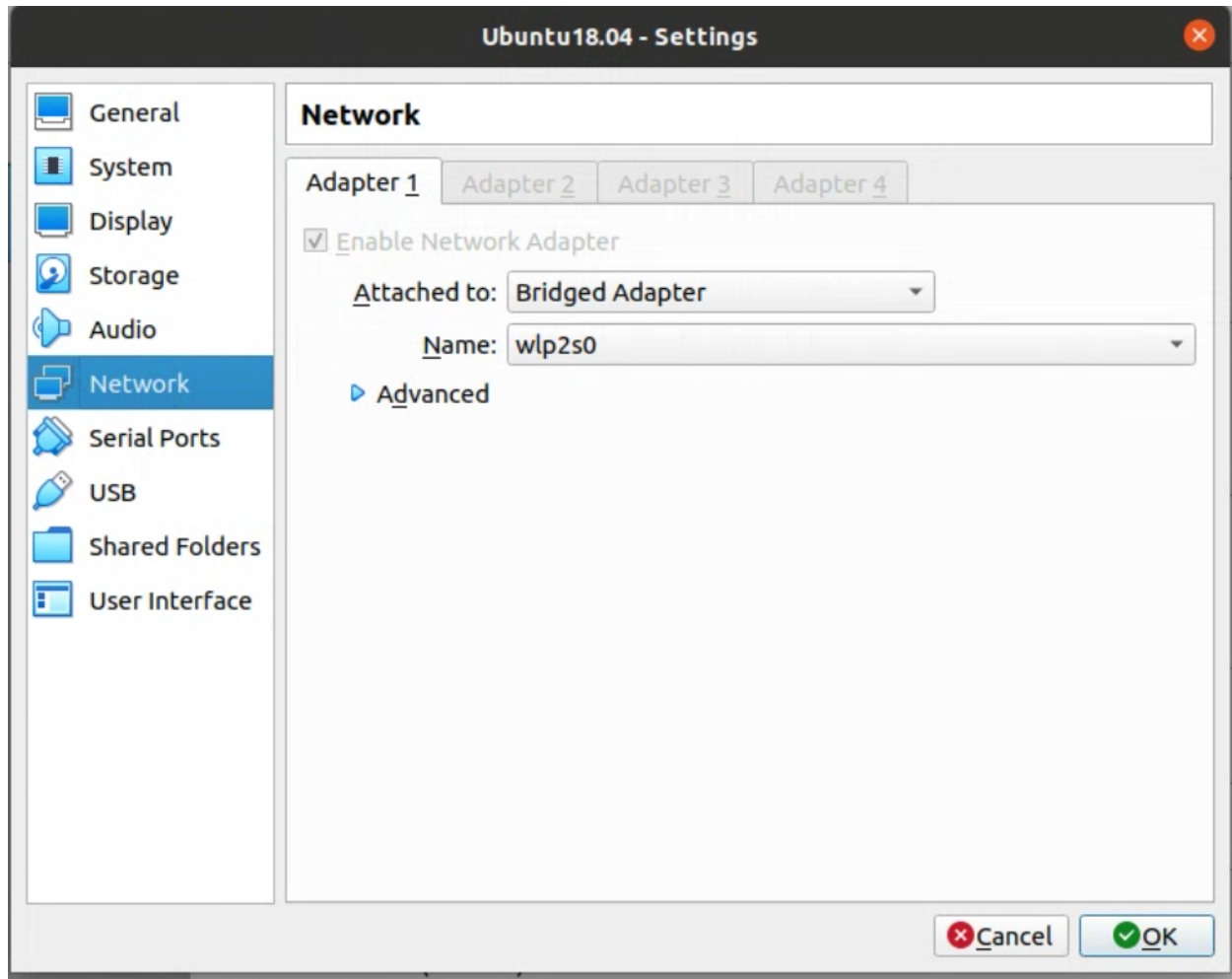
Demo - Ansible with Shell Commands

Setup

Download [VirtualBox](#) and [18.04 Ubuntu Server image](#)

[Setup Ubuntu 18.04 VM](#)

Change the Network adapter of the created VM in settings,



Run the following command on the VM,
`sudo apt-get install openjdk-8-jdk`

Install openssh-server on your host machine (not VM),
`sudo apt install openssh-server`

On Host Machine,
`ssh-keygen -t rsa`

Find the IP address of the remote user (VM) using the command `ifconfig` on the remote user's terminal.

`ssh-copy-id REMOTE_HOSTNAME@REMOTE_IP_ADDRESS`
For example, `ssh-copy-id jasvin@172.16.134.181`

Now you can test whether everything works by ssh-ing to the remote user,
`ssh REMOTE_HOSTNAME@REMOTE_IP_ADDRESS`

Still on host machine,
sudo apt update
sudo apt install ansible
ansible --version

Actual Demo

Inventory file is used to specify the list of managed hosts/server machines.
Example Inventory file,

```
[ubuntu18]  
172.16.134.181 ansible_user=jasvin
```

Create this file in the root directory of your project repository with the name “inventory”

ubuntu18 is the name that we are assigning to the managed host.

So from now on, the host machine (where Ansible is installed) will recognize the specified remote machine as *ubuntu18*.

The IP Address specified is the IP Address of the remote machine and *jasvin* is the username of the account on that remote machine that will serve as the Ansible user.

Run Adhoc ping command

ansible ubuntu18 -m ping -i inventory

The *-i* option is used to specify a different inventory location to use than the default in the current Ansible configuration file.

The *-m* option takes as an argument the name of the module that Ansible should run on the targeted hosts. Modules are small programs that are executed to implement your task.

Ansible facts are data related to your remote systems, including operating systems, IP addresses, attached filesystems, and more.

Ansible commands can be run ad-hoc (from CLI) or via playbooks.

Playbooks

The main purpose of creating playbooks is that it encapsulates all the tasks under one playbook file. Playbooks are *yaml* files like GitHub Actions are.

Note: *yaml* syntax of Playbooks are very strict, everything matters, the “*---*” at the start, indentation and all that good stuff.

Example Playbook file, *p1.yaml*

```
---
- name: Test connectivity to target servers
  hosts: ubuntu18
  tasks:
    - name: Ping test
      ping:
```

Run the *yml* file using the command,
ansible-playbook p1.yml -i inventory

Create a playbook *p2.yml* (to demonstrate copy module)

```
---
- name: Deploy a jar file
  hosts: ubuntu18
  remote_user: vaibhav
  tasks:
    - name: Copy jar file to home directory of remote machine
      copy:
        src:
          ./target/calculatorDevOps-1.0-SNAPSHOT-jar-with-dependencies.jar
        dest: ~/
```

“./target/calculatorDevOps-1.0-SNAPSHOT-jar-with-dependencies.jar” = Relative path of JAR file in the host machine.

Run the *yml* file
ansible-playbook p2.yml -i inventory

Run *.jar* file on Linux Guest (VM/Managed Host)

java -jar calculatorDevOps-1.0-SNAPSHOT-jar-with-dependencies.jar

```
Ubuntu 18 Live Server [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
* Super-optimized for small spaces - read how we shrank the memory
  footprint of MicroK8s to make it the smallest full K8s around.

  https://ubuntu.com/blog/microk8s-memory-optimisation

31 updates can be applied immediately.
8 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

New release '20.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

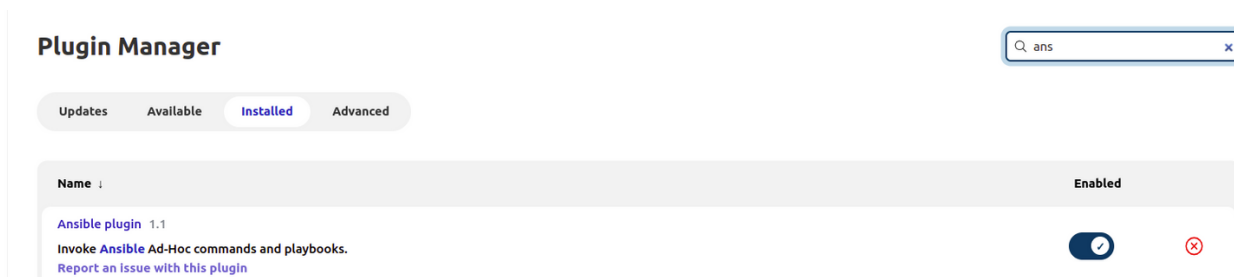
vaibhav@managed_host1:~$ java -jar calculatorDevOps-1.0-SNAPSHOT-jar-with-dependencies.jar
Calculator-DevOps, Choose to perform operation
Press 1 to find factorial
Press 2 to find Square root
Press 3 to find power
Press 4 to find natural logarithm
Press 5 to exit
Enter your choice: 1
Enter a number : 3
16:42:09.360 [main] INFO calculator.Calculator - [FACTORIAL] - 3.0
16:42:09.378 [main] INFO calculator.Calculator - [RESULT - FACTORIAL] - 6.0
Factorial of 3.0 is : 6.0

Calculator-DevOps, Choose to perform operation
Press 1 to find factorial
Press 2 to find Square root
Press 3 to find power
Press 4 to find natural logarithm
Press 5 to exit
Enter your choice: 5
Exiting....
vaibhav@managed_host1:~$ _
```

Demo - Ansible Using Jenkins

Step 1: Installing Ansible plugin on Jenkins

First we have to set up the plugin for Ansible. You can add ansible plugin in jenkins, under Manage Jenkins → Manage Plugins , here search for ansible plugin in available , select Ansible and click on Install without restart



Step 2: Setting up Git and Ansible on Jenkins

First check the path for git and ansible on your machine by typing the following commands on the terminal

```
$ which git
```

```
$ which ansible
```

Now go to Manage Jenkins→Global Tool Configuration

Add Git and Ansible path as shown below

Git

Git installations

Git

Name

git

Path to Git executable ?

/usr/bin/git

☐ Install automatically ?

Delete Git

Add Git ▾

Ansible

Ansible installations

Add Ansible

Ansible

Name

Ansible

Path to ansible executables directory

/usr/bin/

☐ Install automatically ?

Delete Ansible

Add Ansible

List of Ansible installations on this system

Save

Apply

Click on Save

Step 3: Adding GitHub credentials to Jenkins

1. Login to your GitHub account
2. Go to Settings→Developer Settings→Personal Access Tokens and then click on Generate New Token
3. Given name to your token, select expiration period (make sure to select more than 1 month)
4. Select the following checkboxes

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☐ write:packages	Upload packages to GitHub Package Registry
☐ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ write:org	Read and write org and team membership, read and write org projects
☒ read:org	Read org and team membership, read org projects
☐ admin:public_key	Full control of user public keys
☐ write:public_key	Write user public keys
☐ read:public_key	Read user public keys
☐ admin:repo_hook	Full control of repository hooks
☐ write:repo_hook	Write repository hooks
☐ read:repo_hook	Read repository hooks
☐ admin:org_hook	Full control of organization hooks
☐ gist	Create gists
☐ notifications	Access notifications
☐ user	Update ALL user data
☐ read:user	Read ALL user profile data
☒ user:email	Access user email addresses (read-only)

5. Click on Generate Token. Keep the token in a save place
6. Now again go to Jenkins→Manage Jenkins→Manage Credentials→Jenkins (under Store column)→Global Credentials→Click on Add Credentials (On the left Side)

The screenshot shows the Jenkins web interface. At the top, there's a header with the Jenkins logo, a search bar, and user information (Vaibhav Tandon). Below the header, a breadcrumb trail reads: Dashboard > Credentials > System > Global credentials (unrestricted). On the left sidebar, there are two links: 'Back to credential domains' and 'Add Credentials'. The main content area is the 'Add Credentials' form. It has a 'Kind' dropdown set to 'Username with password'. Below it is a 'Scope' dropdown set to 'Global (Jenkins, nodes, items, all child items, etc)'. There are input fields for 'Username', 'Password', 'ID', and 'Description'. A checkbox labeled 'Treat username as secret' is present. At the bottom of the form is an 'OK' button.

Under Username add your Github Username, under password add your access token (generated earlier), under ID give name to your credentials and provide a simple description.

7. Click OK and you have successfully added your GitHub credentials to your Jenkins account

Step 4: Adding ssh keys to Jenkins

Now in order to access remote server via Jenkins type the following commands

```
$ sudo su - jenkins
$ ssh-keygen -t rsa
$ ssh-copy-id REMOTEUSER@<REMOTE IP ADDRESS>
```



```

vaibhav@vaibhav-X541UJ:~/Desktop$ sudo su jenkins
[sudo] password for vaibhav:
jenkins@vaibhav-X541UJ:/home/vaibhav/Desktop$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/var/lib/jenkins/.ssh/id_rsa):
Created directory '/var/lib/jenkins/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /var/lib/jenkins/.ssh/id_rsa
Your public key has been saved in /var/lib/jenkins/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:KEkj/ezd6nA+9uvPSREy6bgQui6gX2Yxi0fsYfv46ts jenkins@vaibhav-X541UJ
The key's randomart image is:
+---[RSA 3072]-----+
|
| . + . + .
| + * o o o .
| @ = S . .
| . = X o o .
| ... X o + . .
| . * + + o . o .
| ..o*+Eo+++o+
+---[SHA256]-----+
jenkins@vaibhav-X541UJ:/home/vaibhav/Desktop$ vaibhav@192.168.1.2
vaibhav@192.168.1.2: command not found
jenkins@vaibhav-X541UJ:/home/vaibhav/Desktop$ ssh-copy-id vaibhav@192.168.1.2
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/var/lib/jenkins/.ssh/id_rsa.pub"
The authenticity of host '192.168.1.2 (192.168.1.2)' can't be established.
ECDSA key fingerprint is SHA256:Hqg0RyR4Hea4ZLw2CqRN2vxRCLmJN/rV4LIaBaDorA.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
vaibhav@192.168.1.2's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'vaibhav@192.168.1.2'"
and check to make sure that only the key(s) you wanted were added.

jenkins@vaibhav-X541UJ:/home/vaibhav/Desktop$

```

This will copy the remote user's ssh keys to the known hosts in our jenkins ssh directory and now jenkins will be able login to the remote server without any need of the password, and we can run our ansible playbook without any password needed.

Step 5: Creating and running the Jenkins Pipeline

1. Go to Jenkins Dashboard→New Item→ Select Pipeline and give it a name→Click OK
2. Inside your pipeline click on Configure (On the left side)
3. Add the following Pipeline Declarative Syntax as shown below:

```

pipeline {
    // The "agent" section configures on which nodes the pipeline can be
    // run. Specifying "agent any" means that Jenkins will run the job on
    // any of the available nodes.
    agent any
    stages {
        stage('Git Pull') {
            steps {
                // Get code from a GitHub repository
                // Make sure to add your own git url and credentialsId
                // (GitHub PAT) which is added to Jenkins under Manage
            }
        }
    }
}

```

```

        // Credentials
        git url:
        '<Git Link to your GitHub repo>',
        branch: 'main',
        credentialsId: '<Your Credentials ID added to Jenkins>'
    }
}
stage('Maven Build') {
    steps {
        // Maven build
        sh 'mvn clean install'
    }
}
stage('Ansible Deploy') {
    steps {
        // Ansible Deploy to remote server (managed host)
        ansiblePlaybook colored: true,
        disableHostKeyChecking: true,
        installation: 'Ansible',
        inventory: 'inventory',
        playbook: 'p2.yml'
    }
}
}
}
}

```

GeneralBuild TriggersAdvanced Project OptionsPipeline

Pipeline

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   stages {
4     stage('Git Pull') {
5       steps {
6         // Get code from a GitHub repository
7         git url: 'https://github.com/vtandonv/CalculatorUsingDevOps.git', branch: 'main',
8           credentialsId: 'jenkins_demo'
9       }
10    }
11    stage('Maven Build') {
12      steps {
13        // Maven build
14        sh 'mvn clean install'
15      }
16    }
17    stage('Ansible Deploy') {
18      steps {
19        ansiblePlaybook colored: true, disableHostKeyChecking: true, installation: 'Ansible', inventory: 'inventory', playbook: 'p2.yml'
20      }
21    }
22  }
23 }
```

☒ Use Groovy Sandbox ?

Pipeline Syntax

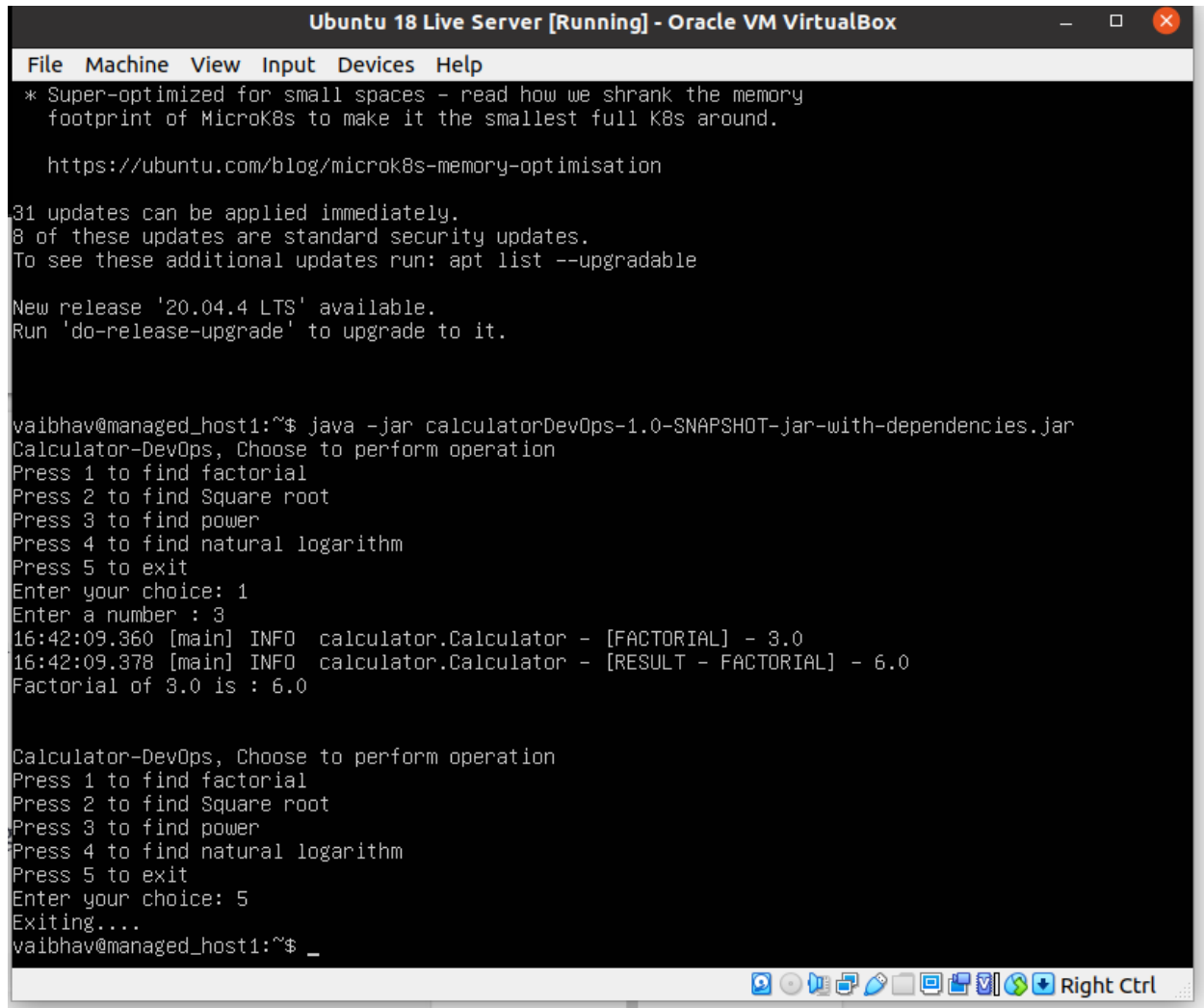
Save

Apply

- Click on Save and then click on Build Now to build the pipeline. It will take a few minutes to build

Step 6: Running .jar file on remote user

Now login to your remote user and run your .jar file



```
Ubuntu 18 Live Server [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
* Super-optimized for small spaces - read how we shrank the memory
  footprint of MicroK8s to make it the smallest full K8s around.

  https://ubuntu.com/blog/microk8s-memory-optimisation

31 updates can be applied immediately.
8 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

New release '20.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

vaibhav@managed_host1:~$ java -jar calculatorDevOps-1.0-SNAPSHOT-jar-with-dependencies.jar
Calculator-DevOps, Choose to perform operation
Press 1 to find factorial
Press 2 to find Square root
Press 3 to find power
Press 4 to find natural logarithm
Press 5 to exit
Enter your choice: 1
Enter a number : 3
16:42:09.360 [main] INFO calculator.Calculator - [FACTORIAL] - 3.0
16:42:09.378 [main] INFO calculator.Calculator - [RESULT - FACTORIAL] - 6.0
Factorial of 3.0 is : 6.0

Calculator-DevOps, Choose to perform operation
Press 1 to find factorial
Press 2 to find Square root
Press 3 to find power
Press 4 to find natural logarithm
Press 5 to exit
Enter your choice: 5
Exiting....
vaibhav@managed_host1:~$ _
```

Hence using Ansible and Jenkins pipeline we were able to deploy .jar file to our remote server

So basically, we fetched the repo from GitHub, built it using Maven which gives us the deployable JAR file. We then use the Ansible Playbook to copy this JAR file to the home directory of the managed hosts (in our case, only a single machine whose address we specified in the inventory file) and now the managed host machine is ready to run the JAR file without requiring any of the dependencies installed, nor did it need to actually build the application. It can run as it is, just needs the core library (in this case, Java) installed.

You can get help with setting up Ansible pipeline scripts by clicking the “Pipeline Syntax” button in the Jenkins project page.

“This **Snippet Generator** will help you learn the Pipeline Script code which can be used to define various steps. Pick a step you are interested in from the list, configure it, click **Generate Pipeline Script**, and you will see a Pipeline Script statement that would call the step with that configuration. You may copy and paste the whole statement into your script, or pick up just the

options you care about. (Most parameters are optional and can be omitted in your script, leaving them at default values.)”

Demo - Ansible + Jenkins + Docker

On Host

`docker pull ubuntu`

`docker create --name containerName ubuntu`

`docker start containerName`

`docker attach containerName`

In Docker container,

Create new user in container as,

`useradd -m userName`

`sudo passwd userName`

`usermod -aG sudo userName`

On Host,

Install

`openssh-server: sudo apt install openssh-server`

`sshpass: sudo apt install sshpass`

`sudo service ssh restart`

In Docker container

Install

`openssh-server: sudo apt install openssh-server`

Note: If this doesn't work then try *`apt-get update`* first

`sudo service ssh restart`

Check IP Address using, “cat /etc/hosts” in root user

On Host,

`ssh-keygen -t rsa`

`ssh-copy-id REMOTEUSER@<REMOTE IP ADDRESS>`

On Jenkins,

Configure it the same as Ansible + Jenkins

In the Ansible + Jenkins repo <https://github.com/vtandonv/CalculatorUsingDevOps>

Update inventory file as,

[ubuntu18]

172.17.0.2 ansible_user=jasvin

[docker]

172.17.0.2 ansible_ssh_pass=password ansible_ssh_user=jasvin

ansible_ssh_pass = Ansible user password on remote machine

Update p2.yml as,

--

- name: Deploy a jar file

hosts: **docker**

remote_user: **jasvin**

tasks:

- name: Copy jar file to home directory

copy:

src: ./target/calculatorDevOps-1.0-SNAPSHOT-jar-with-dependencies.jar

dest: ~/

Now you should be able to build the job.

Don't forget: If SSH ever fails then you've most likely forgotten to start the ssh service on the container, *service ssh restart* [on the container]

31/03/22

Continuous Monitoring

What is continuous monitoring?

- Continuous monitoring is a technology and process that IT organizations use to get near-immediate feedback and insight into performance and interactions across the network, which helps drive operational, security and business performance.
- Continuous monitoring, sometimes referred to as ConMon or Continuous Control Monitoring (CCM), provides security and operations analysts with real-time feedback on the overall health of IT infrastructure, including networks and applications deployed in the cloud.

Importance of CCM

- Goal is to increase visibility of working applications and transparency of network activities.
- Could help indicate a security breach, and to mitigate the risk of cyber-attacks with timely alert systems that trigger rapid incident response.

- Detect application performance issues, identify their cause and implement a solution before any adversary occurs.
- Ultimately, provide near immediate feedback and insight into performance and interactions across the network.

Scope of CCM

- Application Monitoring: performance and availability of application services.
- Configuration Monitoring: application and environment-specific configurations for specific app/environment version.
- Database Monitoring: database connection time and locks, cache size, performance, queries (most frequent, top CPU, long running), replication details, CPU/Memory and disk.
- Middleware Monitoring: queue services, communication and data management across app and operating systems, message services.
- Infrastructure Monitoring: virtualization and physical hardware utilization, performance and availability, network latency.
- Third-party Monitoring: services availability, the response time of partner services.
- Batch Monitoring: scheduled jobs start time, duration, completion rate.
- Data Monitoring: data quality, uniformity, accuracy and completeness.
- Security Monitoring: vulnerability management, end-points, data access and authorization, and event monitoring.

ELK Stack

- "ELK" is the acronym for three open-source projects: Elasticsearch, Logstash, and Kibana.
- Elasticsearch is a search and analytics engine.
- Logstash is a server-side data processing pipeline that ingests data from multiple sources simultaneously, transforms it, and then sends it to a "stash" like Elasticsearch.
- Kibana lets users visualize data with charts and graphs in Elasticsearch.
- Elasticsearch is an open source, full-text search and analysis engine, based on the Apache Lucene search engine. Logstash is a log aggregator that collects data from various input sources, executes different transformations and enhancements and then ships the data to various supported output destinations. Kibana is a visualization layer that works on top of Elasticsearch, providing users with the ability to analyze and visualize the data. And last but not least — Beats are lightweight agents that are installed on edge hosts to collect different types of data for forwarding into the stack.
- Together, these different components are most commonly used for monitoring,

troubleshooting and securing IT environments (though there are many more use cases for the ELK Stack such as business intelligence and web analytics). Beats and Logstash take care of data collection and processing, Elasticsearch indexes and stores the data, and Kibana provides a user interface for querying the data and visualizing it.

Elasticsearch:

Elasticsearch is an open source, distributed, RESTful, full-text search engine built on top of Apache Lucene. Elasticsearch is developed in JAVA. It uses schema free JSON documents and comes with extensive REST APIs for storing and searching the data.

Whenever in your system you run Elasticsearch an instance of Elasticsearch known as node is created which has a unique id and a name and this node belongs to a single cluster.

You can have several nodes in a cluster where the cluster is assigned to perform a specific goal. Elasticsearch works on the concept of sharding and indexing.

What is Sharding?

Sharding is a database architecture pattern related to horizontal partitioning — the practice of separating one table's rows into multiple different tables, known as partitions. Each partition has the same schema and columns, but also entirely different rows. Likewise, the data held in each is unique and independent of the data held in other partitions.

It can be helpful to think of horizontal partitioning in terms of how it relates to vertical partitioning.

In a vertically-partitioned table, entire columns are separated out and put into new, distinct tables.

The data held within one vertical partition is independent from the data in all the others, and each hold both distinct rows and columns.

Sharding involves breaking up one's data into two or more smaller chunks, called logical shards. The logical shards are then distributed across separate database nodes, referred to as physical shards, which can hold multiple logical shards. Despite this, the data held within all the shards collectively represent an entire logical dataset.

Elasticsearch uses this for scaling purposes and for speeding up the search results replicas of the shards can also be used.

What is Indexing?

Elasticsearch stores data in the form of documents and these documents are contained under indexes.

The index can be thought of as a table in SQL. An Elasticsearch cluster can have multiple indices, which in turn contain multiple types. These types then have multiple documents. A document is similar to a row in SQL. Documents can have multiple fields that provide

information about a data object. Elasticsearch uses an inverted index to achieve a fast full-text search over documents. An inverted index consists of all the unique words in the document, and each unique word is mapped to the list of documents containing it.

For the sake of simplicity let's consider the two documents.

Thor should defeat Hela.

The Avengers must defeat Thanos

The inverted index can be built by splitting the documents into tokens, listing all the unique words and then marking all the documents where the word appears. The inverted index for the above documents looks like.

Term	Document1	Document2
Thor	X	
should	X	
defeat	X	X
Hela	X	
The		X
Avengers		X
must		X
Thanos		X

Kibana

Kibana is an exploration and visualization tool. You can use it to analyze Elasticsearch's logs or time series data and monitor the specific application. Kibana needs to rely on the elasticsearch service. By default, it will listen to the elasticsearch service on port 9200, and start the service on port 5601. (Ofcourse, like Elasticsearch, you can customize these configurations at specific yaml files.) Kibana is a tool to visualize Elasticsearch data in real-time. It provides a variety of visualizations to get the best meaning out of your data. Dashboards can be created according to use cases, and get updated with new data automatically.

Some of the main features of Kibana are:

- Leverages the aggregation capabilities of Elasticsearch
- Has many predefined visualizations and it is very easy to define new ones using Vega.
- Can plot complex data like geo-data, time series metrics, relationship graphs, etc
- Unsupervised machine learning has been added to detect anomalies in Elasticsearch data
- Dashboards can be made and transported in the form of PDF, PNG or level-controlled links
- Alerts and notifications can be set up for any specific query

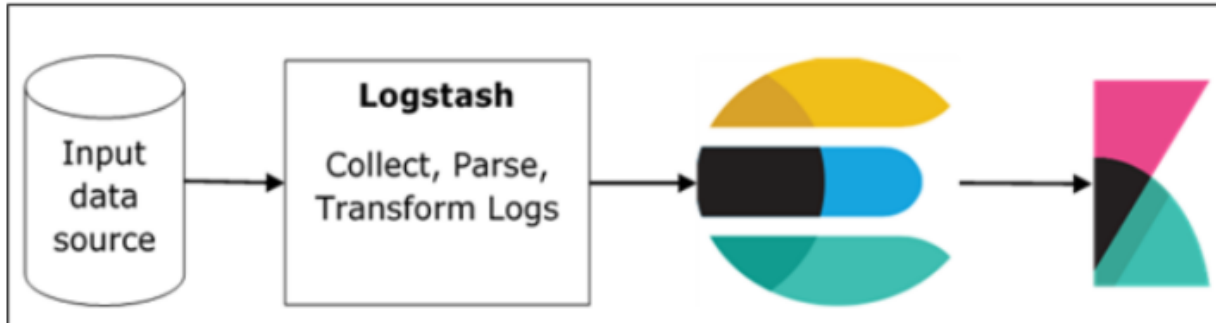
Logstash

Logstash is a tool based on the filter/pipes patterns for gathering, processing and generating the logs or events. It helps in centralizing and making real time analysis of logs and events from different sources.

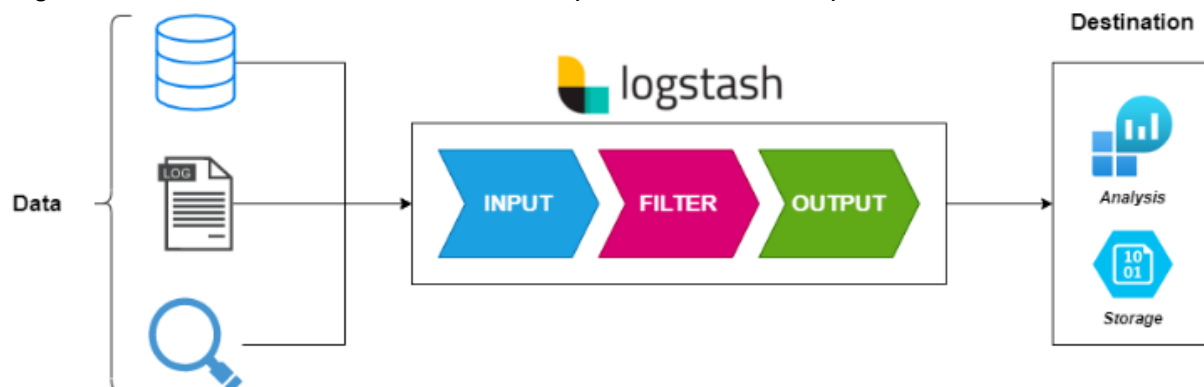
Logstash is written on JRuby programming language that runs on the JVM, hence you can run Logstash on different platforms. It collects different types of data like Logs, Packets, Events, Transactions, Timestamp Data, etc., from almost every type of source. The data source can be Social data, E-commerce, News articles, CRM, Game data, Web trends, Financial data, Internet of Things, Mobile devices, etc.

The general features of Logstash are as follows –

- Logstash can collect data from different sources and send to multiple destinations.
- Logstash can handle all types of logging data like Apache Logs, Windows Event Logs, Data over Network Protocols, Data from Standard Input and many more.
- Logstash can also handle http requests and response data.
- Logstash provides a variety of filters, which helps the user to find more meaning in the data by parsing and transforming it.
- Logstash can also be used for handling sensors data in internet of things.
- Logstash is open source and available under the Apache license version 2.0.



Logstash consists of three core sections—inputs, filters, and outputs—as shown here:



Logstash inputs

The primary feature of Logstash is its ability to collect and aggregate data from multiple sources. With over 50 plugins that can be used to gather data from various platforms and services, Logstash can cater to a wide variety of data collection needs from a single service. These inputs range from common inputs like file, beat, Syslog, stdin, UDP, TCP, HTTP, heartbeat to specific services such as Azure event hub, Apache Kafka, AWS Kinesis, Salesforce, and SQLite. Inputs are the starting point of Logstash configuration. By default, Logstash will automatically create a stdin input if there are no inputs defined. One of the most important considerations is that users will need to properly configure, categorize, and tag the inputs so that the data can be properly processed (filtered) and sent to its required output destination.

Logstash filters

What distinguishes Logstash from most other services is its ability to apply filters to the input data and process it. Unlike acting as a simple aggregator and pushing data, Logstash extracts information from raw data and transforms it into more meaningful common formats as an intermediary step before sending it for further processing or storage.

This process not only reduces the workload of further processing services like Elasticsearch; it also provides a common format that can be easily processed for better analysis of the gathered data. Logstash provides multiple filter plugins from a simple CSV plugin to parse CSV data to grok, allowing unstructured data to be parsed into fields to urlencode, etc.

Logstash outputs

The final part of Logstash is its output. As mentioned earlier, Logstash can output the collected (input) and processed (filter) data into a variety of outputs from Elasticsearch itself to simple files, storage services like S3, messaging services like SQS, and Kafka to other services like AWS CloudWatch and Google BigQuery.

All these plugins lead Logstash to become one of the most versatile solutions for gathering data. Outputs are not limited to a single destination. By default, Logstash creates a stdout output if no outputs are configured. However, Logstash can be configured to push data to multiple destinations while filtering out specific inputs to specific outputs.

Filter Stage - Grok Plugin

It helps you to define a search and extract parts of your log lines into structured fields.

Plugins available are:

<https://github.com/logstash-plugins/logstash-patterns-core/tree/master/patterns>

Grok works by combining text patterns into something that matches your logs.

Here is the basic syntax format for a Logstash grok filter:

```
grok {  
  match => { "message" => "%{SYNTAX:SEMANTIC}" }  
}
```

The SYNTAX will designate the pattern in the text of each log. The SEMANTIC will be the identifying mark that you actually give that syntax in your parsed logs. In other words: This will match the predefined pattern and map it to a specific identifying field.

```
%{PATTERN:FieldName}
```

For example, a pattern like 127.0.0.1 will match the Grok IP pattern, usually an IPv4 pattern. Grok has separate IPv4 and IPv6 patterns, but they can be filtered together with the syntax IP. Since grok is essentially based upon a combination of regular expressions, you can also create your own custom regex-based grok filter with this pattern:

```
(?<custom_field>custom pattern)
```

You can debug your grok expression here at this link –

<https://grokconstructor.appspot.com/do/match>

Here's a sample grok for a sample log

```
grok {  
  match => {  
    "message" => '%{IPORHOST:clientip} %{USER:ident} %{USER:auth}  
\\[%{HTTPDATE:timestamp}\\] "%{WORD:verb} %{DATA:request}  
HTTP/%{NUMBER:httpversion}" %{NUMBER:response:int}  
(?:-|%{NUMBER:bytes:int}) %{QS:referrer} %{QS:agent}'  
  }  
}
```

05/04/22

Dockerfile

Instructions

FROM - It is the base image of your docker image. Whenever possible, use current official images as the basis for your images for it to intake subsequent instructions.

```
FROM ubuntu:20.04
```

LABEL - You can add labels to your image to help organize images by project, record licensing information, to aid in automation, or for other reasons. For each label, add a line beginning with LABEL and with one or more key-value pairs.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
```

RUN - The RUN instruction will execute any commands in a new layer on top of the current image and commit the results.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
```

CMD - The CMD instruction should be used to run the software contained in your image, along with any arguments. CMD should almost always be used in the form of CMD [*"executable", "param1", "param2"...*]. There can only be one CMD instruction in a Dockerfile. If you list more than one CMD then only the last CMD will take effect.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
```

EXPOSE - The EXPOSE instruction indicates the ports on which a container listens for connections.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
EXPOSE 80
```

ENV - The ENV instruction is also useful for providing required environment variables specific to services you wish to containerize.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
EXPOSE 80
ENV PG_MAJOR=9.3
```

ADD or COPY - Although ADD and COPY are functionally similar, generally speaking, COPY is preferred. That's because it's more transparent than ADD. COPY only supports the basic copying of local files into the container, while ADD has some features (like local-only tar extraction and remote URL support) that are not immediately obvious. Consequently, the best use for ADD is local tar file auto-extraction into the image, as in *ADD rootfs.tar.xz /*

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
EXPOSE 80
ENV PG_MAJOR=9.3
COPY requirements.txt /tmp/
ADD https://example.com/big.tar.xz /usr/src/things/
```

ENTRYPOINT - An ENTRYPOINT allows you to configure a container that will run as an executable.

WORKDIR - The WORKDIR instruction sets the working directory for any RUN, CMD, ENTRYPOINT, COPY and ADD instructions that follow it in the Dockerfile. If the WORKDIR doesn't exist, it will be created even if it's not used in any subsequent Dockerfile instruction.

```
FROM ubuntu:20.04
LABEL com.example.release-date="2022-02-12"
RUN apt-get install -y curl
CMD echo "This is a test." | wc -
```

```
EXPOSE 80
ENV PG_MAJOR=9.3
COPY requirements.txt /tmp/
ADD https://example.com/big.tar.xz /usr/src/things/
WORKDIR /a/b
```

Example: Build a node application container

```
FROM node:12-alpine
RUN apk add --no-cache python2 g++ make
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

Some further points.

docker <TAB-to-Autocomplete>

On multiple tab presses, will show you a list of commands that can be run after the docker keyword.

docker container ls

Show all active containers

docker container ls -a

Show all containers

docker build -t <tag_name> <path_where_dockerfile_is_stored>

When this Docker image is built, it will be assigned a tag as tag_name

If you are currently in the same directory as the Dockerfile for this image, then you can just replace path with “.”

So the path can be absolute or relative.

docker build -t jasvin/docker_devops .

docker image ls

Show all the pulled images

docker history image_name

Shows you the commands that have been run in order to create the image. Each command forms its own layer on top of the previous image. So each stage shown in the output of history

command is its own layer and on top of it we are running the next command and so on till the final image that we had pulled is ready.

For example, *docker history ubuntu*

We do this same thing with the images that we create as well, we pull the Ubuntu image, create a container using it, then install some packages on that container and each added package thus forms a new layer on top of the base image layer, i.e. Ubuntu (should be ringing some bells to what we had learned in SPE).

Note that when a new layer is being added on top of the previous one, we are entering the container into a new state, so while the addition process is going on, the previous state of the container is just an intermediate container that will be removed once the addition of the new layer is complete and then that layer will encapsulate the entire container.

Run *docker history* on an image that you built and see the magic in action.

<container_id>

While specifying container ids, you only need to specify the unique prefix of the id amongst all other container ids and the command will still work. No need to type out the entire id.

Copy file from host machine to container

docker container cp path_to_file <container_name/container_id>:<absolute_path>

Example,

docker container cp hello.txt 69420:/

69420 = Unique prefix of the container id

hello.txt = Relative file path of the txt file on host machine

/ = Absolute file path of the copied file on the container

Viceversa is also possible,

docker container cp 69420:/hello.txt newhello

For help on commands use the help option, like,

docker container --help

Mapping Host Port to a Container Port

docker run -d -p 9000:80 httpd

Create a container as a daemon (option -d) with the Apache server base image (httpd) and map port 9000 requests on the host machine to port 80 on the container which is Apache's default port.

Pushing to Docker Hub

To push an image, we first need to create a repository on Docker Hub.

1. Sign up or Sign in to Docker Hub. <https://hub.docker.com/>

2. Click the Create Repository button.
3. Name your repo. Make sure the Visibility is Public.
4. Click the Create button!

Next, build your Docker image using the docker build command (which means you must have a Dockerfile and all), assign it a tag in the format, `<username/dockerhub_repository:tagname>` name.

tagname is usually used to specify versions like Ubuntu 20, 18, etc.
If no *tagname* is specified then it is considered to be the latest version.

Now login to Dockerhub using, *docker login*

After this you can do,
docker push <username/dockerhub_repository:tagname>

Note: While automating the Docker build process, if you find a lot of images tagged with “<none>”, then this link should help you out,
<https://stackoverflow.com/questions/48745936/many-none-images-created-after-build-a-docker-image>

07/04/22

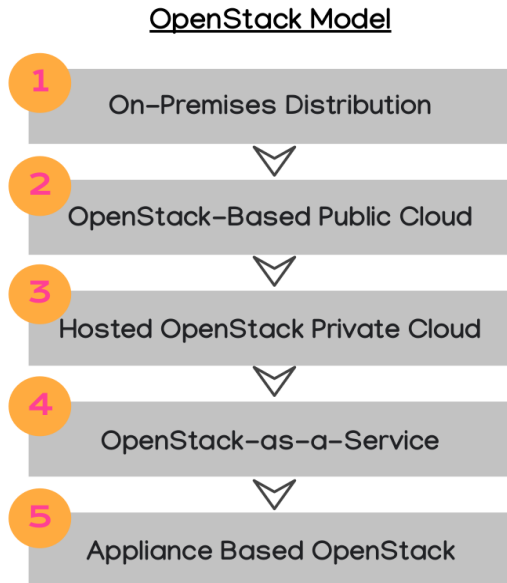
OpenStack

[Copied from CC]

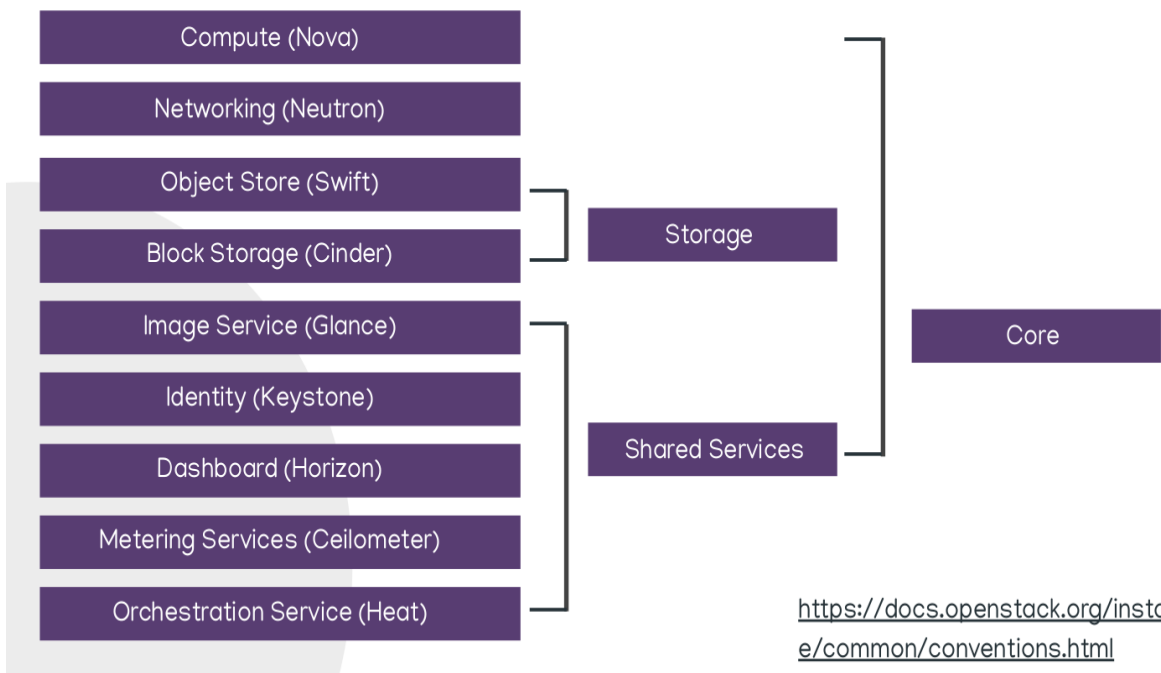
OpenStack is a free open standard cloud computing platform, mostly deployed as infrastructure-as-a-service in both public and private clouds where virtual servers and other resources are made available to users .

OpenStack is a cloud operating system that controls large pools of compute, storage, and networking resources throughout a datacenter, all managed and provisioned through APIs with common authentication mechanisms. A dashboard is also available, giving administrators control while empowering their users to provision resources through a web interface.

OpenStack believes in open source, open design, and open development; all in an open community so anyone can participate.



OPENSTACK SERVICES



<https://docs.openstack.org/install-guide/common/conventions.html>

Keystone

Keystone is an OpenStack service that provides API client authentication, service discovery, and distributed multi-tenant (role-based) authorization by implementing OpenStack identity API.

- Common authentication system
- Central catalog of services and endpoints

- Supports LDAP, AD, MYSQL
- Provides a token for subsequent auth requests.

Provides 4 types of services,

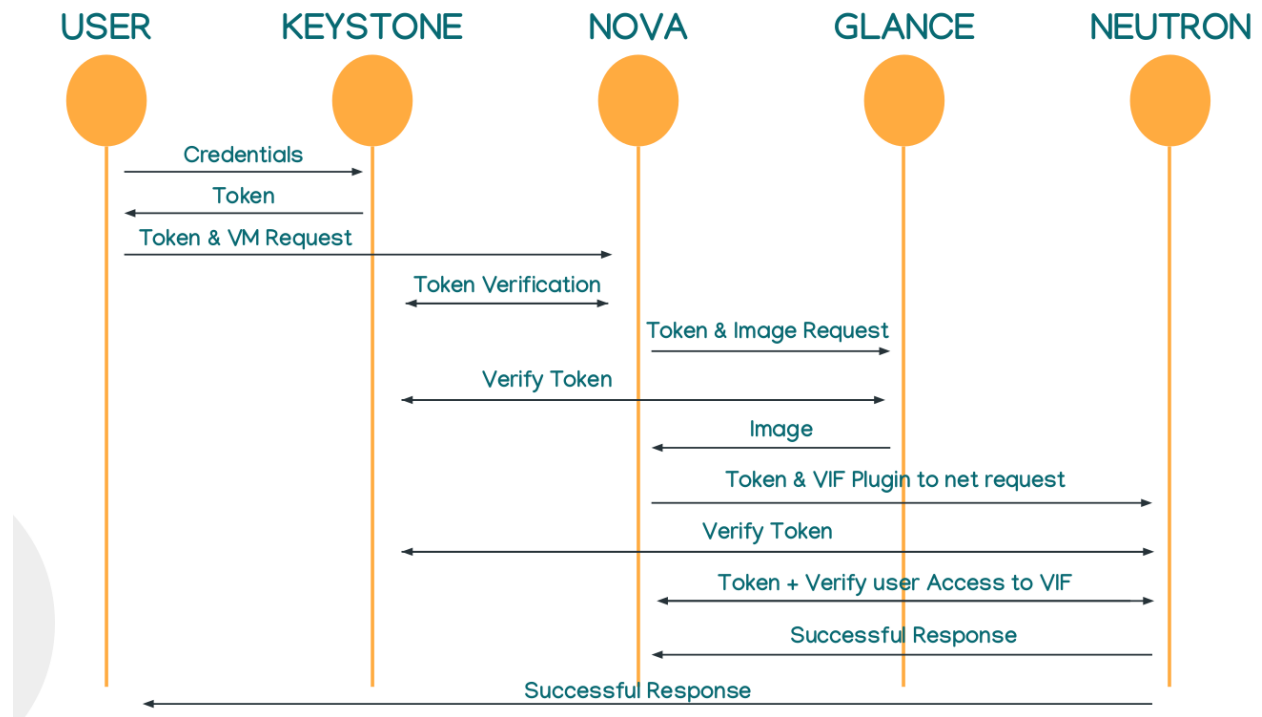
Token Service

Catalog Service

Policy Service

Assignment Service

KEYSTONE - PROCESS FLOW



Glance

- The Glance project provides a service where users can upload and discover data assets.
- This currently includes image and metadata information.
- Glance image service includes discovering, registering and retrieving virtual machine images.

Nova

- Nova is the OpenStack project that provides a way to provision compute instances (aka virtual servers).
- Nova supports creating virtual machines, bare metal servers (through the use of ironic), and has limited support for system containers.
- It runs as a set of daemons on top of existing Linux servers to provide that service.

- It is a component within the OpenStack cloud computing platform developed to provide on-demand access to compute resources by provisioning and managing large networks of virtual machines.
- Nova works with Vmware, Xen, Kvm, Hyper-v and other virtualization technologies.

Neutron

- Neutron is an OpenStack project to provide “Network Connectivity as a Service” between interface devices (e.g., vNICs) managed by other OpenStack services (e.g., Nova).
- It implements the OpenStack Networking API.
- Features,
 - Network, Subnet and Port abstractions.
 - Plugins support multiple technologies.
 - Modular architecture.
 - Central or distributed environment.
 - Accepts API requests and routes them to appropriate plugins for action.
 - Neutron can be scaled to thousands of nodes and networks
 - Auto allocation for creating networking topologies.
 - Ability to logically separate provisioning and tenant networks.

Swift

Swift is a highly available, distributed, eventually consistent object/blob store. Organizations can use Swift to store lots of data efficiently, safely, and cheaply. Object storage service for OpenStack .

Swift proxy is responsible for tying together the swift of rest architecture. For each request, it looks up the container, account or object.

Cinder

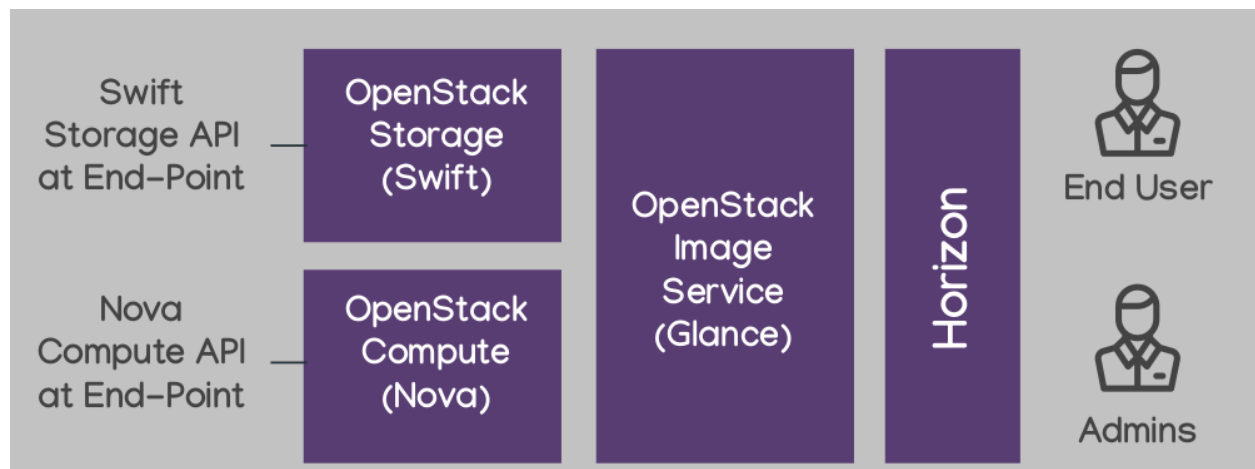
Cinder is the OpenStack Block Storage service for providing volumes to Nova Virtual Machines, Ironic Bare-metal hosts, containers and more.

Some of the goals of Cinder are to be/have,

- Component-based Architecture: Quickly add new behaviors
- Highly available: Scale to very serious workloads
- Fault-tolerant: Isolated processes avoid cascading failures
- Recoverable: Failures should be easy to diagnose, debug, and rectify
- Open Standards: Be a reference implementation for a community-driven API

Horizon

- Horizon is the canonical implementation of OpenStack Dashboard, which provides a web-based user interface to OpenStack services including Nova, Swift, Keystone, etc.
- It is based on the Django model called Django–OpenStack.



Ceilometer

- Ceilometer is a data collection service that provides the ability to normalize and transform data across all current OpenStack core components with work underway to support OpenStack components.
- Its data can be used for billing, resource tracking and alarming capabilities for all OpenStack services.

Heat

Heat is the main project in the OpenStack orchestration program . It implements an orchestration engine to launch multiple composite cloud applications based on templates in the form of text files that can be treated like code.

Add OpenStack ARCHITECTURE DIAGRAM (the simple one)

Continuous Deployment

Who needs CD?

- Where application feature set is a business driver in a competitive market and frequent updates are encouraged.
- Mobile apps, Internet Banking Applications, Retail Ecommerce Websites and LMS System

An organization that adopted micro services on elastic automated infrastructure with blue green deployment methods to ensure stability and availability definitely need CD to manage it.

Each microservice can be its own deployable module that can definitely use the help of CD.

Who doesn't need CD?

When the application is a large monolithic complex code with very high chances of getting broken due to frequent modifications (Complex SAP implementations).

When a business does not want to release every feature just as soon as they are ready but want to wait for certain reasons (Flipkart Big Billion Day Sale). In this case you want Continuous Delivery and not Deployment.

When ITIL process supersedes DevOps process and mandates production releases to only follow a release calendar for ensuring better control on stability.

When there are production environmental changes which can not be automated but are dependencies of the application to be deployed.

- Once the code is ready to deploy, being capable to do the deployment of the same every time to the production is the capability known as **continuous delivery**.
- It's a capability of a team to always be ready for production deployment of the newly checked in code.
- When you always do automated progression of deployment to production environment – this practice is called **Continuous Deployment**.

[Stuff related to containers, just reread the pre-midterm stuff on this topic.]

Roll Back

- When you do the deployment using infrastructure as code then the rollback has to be coded as well – there is no automatic rollback unless you code it in.
- So let's say you are deploying a new version of apache, so after installation you need to run tests and if tests fail then Jenkins has to call a clean up code again written in Puppet/Chef/Ansible that will revert changes and restore previous version of infrastructure as code.
- However if you use Deployment tools like Urbancode or XL Deploy, then the rollback is automatically taken care of in many cases. However, like in any rollback, you may still need to implement some algorithm for rollback even then.

Blue-Green Deployment

Blue-green deployment is a release technique that reduces downtime and risk by running two identical production environments called Blue and Green.

At any time, only one of the environments is live, with the live environment serving all production traffic.

For this example, Green is currently live and Blue is idle.

Green is always active and Blue is the passive one.

If you want to release a new version, first deploy it in the Blue environment and check whether everything is working properly.

If it is all good, then reroute green environment traffic to blue environment and now the blue environment becomes the new green environment.

If it didn't deploy successfully, then the green environment remains as is while DevOps team tries to fix the blue environment.

12/04/22

Docker Caching => Bursting

Use package.json versioning with Node projects for caching [the installed modules] and improving build times.

What is this actually talking about?

Adding the following sequence of instructions in your Dockerfile

```
# Copy the package.json and package-lock.json files to the "app" folder
COPY package*.json ./

# Install the dependencies mentioned in package.json
RUN npm install

# Copy the local files to the "app" folder
COPY . .
```

Docker Compose => How the yml file is written

Refer to Github repository for more info [search up the reasoning for each feature used in the yml file]

Refer to the haproxy configuration as well.

<https://www.youtube.com/watch?v=6bXSfjwQVlc>

Refer to the ppt to get all the links

19/04/22

Why is Kubernetes required?

Docker doesn't handle situations where container shuts down due to one reason (resource limits exceeded) or another and stuff like that.

That's where container orchestration will help, i.e. Kubernetes.

For example, If a container goes down, then Kubernetes will ensure that it will be spun back up again.

Why is the Microservice architecture important?

Spam bs

[add stuff from the ppt]

Different services (containers) interact with each other in a cloud computing environment with the help of Message Queues.

Kubernetes Architecture

Attach that diagram.

Master-worker model.

Requires two things by default, Docker image and config file which tells Kubernetes what is to be done.

Pod = Smallest unit in K8

Pod is a place where you run containers.

Can run upto approx 150 containers in a pod but it's a good practice to just run one container.

A cluster consists of nodes/kubelets, each node/kubelets consists of pods and each pod can consist of containers.

Kubelets typically are on different machines (physical or VM) and Pods are typically on the same physical or virtual machine.

The config file for k8 defines what all is to be created and how much of each thing is required and things like that.

It contains values in the same format as Ansible playbook but it is specifying key-value pairs because it's just JSON data specified via a YAML file.

Attach that config thingy explanation here.

Add those k8 commands here.
YAML file from repo.

Deployments => Object that ensures that all the specified K8 objects (like containers) are up and running, gracefully updated and shut down as well.
Has its own config YAML file.

Graceful updation => Complete all pending and current tasks, accept no new ones and then deploy new pods and finally delete the old ones.
Same as the Blue-Green Deployment model.

Attach Deployment commands.
YAML file from repo.

Services is a k8 object that enables communication between pods on the same kubelet.
It has its own config YAML file.

Add types of services...
[typo, Nord port = Node port]

Cluster IP Service

For communication between pods you will use Cluster IP service.
It doesn't allow communication to entities outside the k8 cluster.

LoadBalancer is very similar to the Node port service.
What's the difference? Who knows :(

The LoadBalancer service internally maps the ports of multiple replicas that are created for the same container. So say mapping is from 3000 to 80 and there are two replicas, you don't have to worry about which replica is being served at 80, you can just rest assured that everything works properly with this mapping. Actually there is an intermediate port between the host port (80) and the target port (3000) that performs this virtual mapping between replicas, you can check by running the svc command on the deployment.

Attach service yaml (there's two, but atleast one you can put)

Attach demo commands thingy
[somehow explain all that sheet]

Helm is a useful tool for Kubernetes config management.

awesome-k8s repo is also there.

